

最优子结构

动态规划 · 8题 · C++ & Python 双语对照

NQ146 AcWing 2 AcWing 2

AcWing 2

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N= 1007;
// 2维的解法
int n,m;
int f[N][N];
int v[N],w[N];

int main()
{
    cin>>n>>m;//读入N,V
    //读入v,w
    for (int i = 1; i <= n; i++) cin>>v[i]>>w[i];
    //朴素的二维dp
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <=m; j++)
        {
            f[i][j]=f[i-1][j];
            if ( j >= v[i]) //剩余体积大于第i项的体积
                f[i][j] = max(f[i][j],f[i-1][j-v[i]]+w[i]);
        }
    //根据定义，从前n个物品中，选取总体积不超过m的选法集合f[n][m]就是答案
    cout<<f[n][m]<<endl;
}

```

Python

AcWing 2

NQ147 AcWing 3 AcWing 3

AcWing 3

输入

 见原题

输出

 见原题

范围

 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;

int n,m;
int f[N][N];
int v[N],w[N];

int main()
{
    cin >>n >>m;
    for (int i = 1; i <= n; i++) cin>>v[i]>>w[i];

    //完全背包
    // 1.f[i][j] = f[i-1][j];(不选物品i)
    // 2.f[i][j]= max(f[i-1][j], f[i][j-v[i]+w[i])
    (选若干个物品i)
    for (int i = 1; i <= n; i++)
        for (int j=0; j <= m; j ++){
            f[i][j]=f[i-1][j];
            if (j >= v[i])
                f[i][j] = max(f[i-1][j], f[i][j-v[i]]
+w[i]);
        }

    cout<<f[n][m]<<endl;
    return 0;
}

```

Python

AcWing 3

NQ148 AcWing 898 AcWing 898

AcWing 898

输入

 见原题

输出

 见原题

范围

 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507;

int n;
int w[N][N], f[N][N]; //三角矩阵依旧用2维存储

int main()
{
    //读入n, w
    cin >> n;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            cin >> w[i][j];

    //初始化最底层的f[n][j]
    for (int i = 1; i <= n; i++) f[n][i] = w[n][i];

    //用dp从底层往上计算每个f[i][j]的值
    for (int i = n-1; i; i--)
        for (int j = 1; j < n; j++)
            f[i][j] = max(f[i+1][j]+w[i][j], f[i+1][j+1] + w[i][j]);

    cout << f[1][1] << endl;
    return 0;
}

```

Python

AcWing 898

NQ149 AcWing 895

AcWing 895

AcWing 895

输入 见原题

输出 见原题

范围 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
// f[i]定义为以s[i]结尾的最长的子序列
// 集合: 所有以第i个数结尾的上升子序列集合
// 属性: Max 上升子序列长度的最大值
const int N = 10007;
int n;
int a[N], f[N];
//状态计算
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i]; //从1开始初始化

    for (int i = 1; i <= n; i++)
    {
        f[i] = 1; // a[i]只有一个元素, 所以f[i]=1;
        for (int j = 1; j < i; j++) //求i之前的f[j]
            if (a[j] < a[i])
                f[i] = max(f[i], f[j] + 1);
    }

    int res = 1;
    for (int i = 1; i <= n; i++)
        res = max(res, f[i]);
    cout << res << endl;
    return 0;
}

```

Python

AcWing 895

NQ150 AcWing 897 AcWing 897

AcWing 897

输入 见原题**输出** 见原题**范围** 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```

for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
    {
        f[i][j] = max(f[i-1][j], f[i][j-1]);
        if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i-1][j-1] + 1);
    }

cout << f[n][m] << endl;
return 0;

```

Python

AcWing 897

NQ151 AcWing 282 AcWing 282

AcWing 282

输入

见原题

输出

见原题

范围

见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int n;
int s[N]; //前缀和
int f[N][N]; //所有将[i,j]合并成一堆的方案数

int main()
{
    cin>>n;
    for (int i = 1; i <=n; i++) cin>>s[i], s[i] += s[i-1]; //计算前缀和

    //区间DP, 先枚举长度, 后枚举端点
    for (int len = 2; len <= n; len++)
        for (int i = 1; i+len-1 <= n; i++) //区间左端点
        {
            //区间右端点
            int j = i + len - 1;
            f[i][j] = 1e9; //初始化为无穷大
            for (int k = i; k < j; k++) //区间dp, 枚举分隔点
                f[i][j] = min(f[i][j], f[i][k]+f[k+1][j]+s[j]-s[i-1]);
        }

    cout<<f[1][n]<<endl;
    return 0;
}

```

Python

AcWing 282

NQ152 AcWing 902 AcWing 902

AcWing 902

输入

 见原题

输出

 见原题

范围

 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;
const int N = 1007;

int n,m;
char a[N],b[N];
int f[N][N]; //所有將a[i]變為b[i]的操作步驟

int main()
{
    cin>>n>>a+1; //下標從1開始
    cin>>m>>b+1;

    //初始化
    for (int i = 0; i < N; i ++ ) f[i][0] = i; //把a[i]變為空串
    for (int i = 0; i < N; i ++ ) f[0][i] = i; //把空串變為b[i]

    // DP
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = min(f[i-1][j] + 1, f[i][j-1] + 1);
            if (a[i]==b[j]) f[i][j] = min(f[i][j], f[i-1][j-1]);
            else f[i][j] = min(f[i][j], f[i-1][j-1] + 1);
        }

    cout<<f[n][m]<<endl;

    return 0;
}

```

Python

AcWing 902

NQ153 AcWing 901 AcWing 901

AcWing 901

输入

 见原题

输出

 见原题

范围

 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int g[N][N]; // 地图高度
int f[N][N]; // 从[i,j]开始滑的最长的轨迹

int n, m; // 宽高

int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};

int dp(int x, int y)
{
    // 记忆化搜索
    if (f[x][y] != -1) return f[x][y];
    // 否则, 从x,y开始滑
    f[x][y] = 1; // 当前位置至少是1格
    // 分上下左右四个方向滑雪
    for (int i = 0; i < 4; i++)
    {
        int a = x + dx[i], b = y + dy[i];
        if (a < 1 || a > n || b < 1 || b > m) continue;
        // 越界
        if (g[a][b] >= g[x][y]) continue; // 不能走
        // 四个方向深搜, 更新f[a][b], 取最大值
        f[x][y] = max(f[x][y], dp(a, b) + 1);
    }

    return f[x][y]; // 返回f[x][y]
}

int main()
{
    cin >> n >> m;
    // 读入高度图
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> g[i][j];

    memset(f, -1, sizeof f); // 初始化为-1, 表示该点未被访问

    // 把每个点作为起点深搜
    int res = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            res = max(res, dp(i, j));

    cout << res << endl;
    return 0;
}

```

Python

AcWing 901

